Task 1 - Complete Step-by-Step Run Guide

Project: BrightMart Inventory & Orders API (Cloud-Hosted Business API)

Arrowstack Cloud Computing Internship - Project 1

This guide takes you from zero to a working, verified AWS deployment, and back to zero cost. Follow the parts in order.

Every step says what to type, what you should see, and what to do if you see something else.

Part	What you do	Time	Cost
A	Check your laptop tools	5 min	Free
B	Run and test the API locally	15 min	Free
C	Prepare your AWS account and the AWS CLI	20 min	Free
D	Put the code on GitHub	10 min	Free
E	Deploy to AWS	20 min (mostly waiting)	Free tier
F	Verify in the cloud and collect evidence	45 min	Free tier
G	Ship a code change (optional)	5 min	Free tier
H	Tear down so you are not billed	15 min	-
I	Troubleshooting (every common error + fix)	as needed	-

Conventions: every command is typed in **Windows PowerShell** unless stated otherwise. Lines starting with `#` are comments; do not type them. In the PDF version, long commands wrap onto a second line inside the grey box - type them as **one line** (or copy them from `RUN-GUIDE.md`, which has the same content).

Part A - Check your laptop

A1. Open PowerShell in the project folder

1. Open File Explorer and go to `E:\ArrowStack Internship\project1-cloud-business-api`.
2. Click the address bar, type `powershell`, press Enter. A PowerShell window opens already in that folder.
3. Confirm the location:

```
Get-Location
```

Expected: `E:\ArrowStack Internship\project1-cloud-business-api`

A2. Check Python and Git

```
python --version
git --version
```

Expected: `Python 3.12.x` or `3.13.x`, and `git version 2.x`.

Problem	Fix
`python is not recognized` or the Microsoft Store opens	Install Python from python.org, tick **"Add python.exe to PATH"** in the installer, then close and reopen PowerShell
`git is not recognized`	Install Git for Windows from git-scm.com (default options), reopen PowerShell

---

Part B - Run and test locally (no AWS needed)

The virtual environment `.venv` with all packages was already created. If you are on a new machine or deleted `.venv`, do B1 first; otherwise skip to B2.

B1. (Only if `.venv` is missing) Create the environment

```
python -m venv .venv
.venv\Scripts\python -m pip install --upgrade pip
.venv\Scripts\python -m pip install -r requirements-dev.txt
```

Expected: ends without red `ERROR` lines (yellow `notice` lines are fine).

Tip: we call `.venv\Scripts\python` directly so you never need to "activate" the environment. If you prefer activating, run `.venv\Scripts\Activate.ps1`. If that gives *"running scripts is disabled on this system"*, either keep using `.venv\Scripts\python ...` or run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` and answer `Y`.

B2. Run the automated tests

```
.venv\Scripts\python -m pytest -p no:warnings --cov=app
```

Expected (last lines):

```
TOTAL    372    13    97%
35 passed in ...s
```

If any test fails, copy the full red output and send it to me.

B3. Start the API

```
.venv\Scripts\python -m uvicorn app.main:app --reload
```

Expected: lines ending with `Application startup complete.` and JSON log lines such as `{"level": "INFO", "msg": "startup complete", ...}`.

Leave this window open - the API runs as long as it is open.

B4. Use the API in the browser (Swagger UI)

1. Open `http://127.0.0.1:8000/docs` in your browser.
2. Open **GET /health** -> **Try it out** -> **Execute**. Response: `{"status": "ok"}`.
3. Open **POST /products** -> **Try it out**. In the `x-api-key` box type `local-dev-key`. Replace the body with:

```
{"sku": "TSHIRT-BLK-M", "name": "Black T-Shirt (M)", "price_cents": 49900, "stock": 5, "reorder_level": 2}
```

   Execute -> response code **201** with `"id": 1`.

4. Create a second product the same way:

```
{"sku": "CAP-RED", "name": "Red Cap", "price_cents": 19900, "stock": 1, "reorder_level": 2}
```

5. **POST /orders** (x-api-key `local-dev-key`):

```
{"customer_email": "asha@example.com", "items": [{"product_id": 1, "quantity": 2}, {"product_id": 2, "quantity": 1}]}
```

   Response **201**, `total_cents` = 119700.

6. Execute the same order again -> **409** `insufficient_stock` (the cap is sold out). This is the core business rule: no overselling.
7. **GET /products** with `low_stock = true` -> lists the cap (stock 0 <= reorder level 2).
8. Try **POST /products** with a wrong key -> **401**. Try `"price_cents": -5` -> **422**.

Take screenshots of steps 5-7 for your evidence folder (optional but nice for the presentation).

B5. Run the automated smoke test

Open a **second** PowerShell window in the same folder (keep the API running in the first):

```
.venv\Scripts\python scripts\smoke_test.py --base-url http://127.0.0.1:8000 --api-key local-dev-key
```

Expected: 13 lines starting with `PASS` and `13/13 checks passed`. A JSON report is saved in `evidence\`.

B6. Stop and reset

- Stop the API: click the first window and press **Ctrl + C**.
- Reset local data (start fresh): `Remove-Item local.db`

Part B problems

Problem	Cause	Fix
`[Errno 10048] ... only one usage of each socket address`	Port 8000 is already used	Use another port: add `--port 8001` and open `http://127.0.0.1:8001/docs`. Or find the process: `netstat -ano | findstr :8000` then `taskkill /PID <pid> /F`
`ModuleNotFoundError: No module named 'app'`	You are not in the project folder	`cd "E:\ArrowStack Internship\project1-cloud-business-api"`
`No module named uvicorn` / `fastapi`	Packages not installed in `.venv`	Do step B1
pip error building `psycopg`	Old pip	`.venv\Scripts\python -m pip install --upgrade pip` then repeat B1
401 on POST	Missing/wrong key	Type `local-dev-key` in the `x-api-key` field
409 "SKU already exists"	You already created it	Use another SKU or `Remove-Item local.db` and restart the API
Smoke test: `Connection refused`	API not running	Start it (B3) in another window

Part C - Prepare AWS

C1. AWS account

1. Sign in (or sign up) at https://aws.amazon.com/ . Sign-up needs a card for verification; this project stays in the free tier.
2. **Secure the root user**: top-right account name -> **Security credentials** -> **Assign MFA device** (use an authenticator app). Do this before anything else.
3. Note your free-tier type: accounts created **before 15 July 2025** have the classic 12-month free tier; newer accounts have the **Free Plan** (credits for 6 months). Both work for this project.

C2. Create an IAM user for the command line

Never use the root user's keys.

1. Console search bar -> **IAM** -> **Users** -> **Create user**.
2. User name: `intern-deployer`. Do not tick console access (not needed). Next.
3. **Attach policies directly** -> tick **AdministratorAccess** -> Next -> Create user.

   (Needed because the stack creates IAM roles, a VPC, RDS, etc. Delete this user after the internship.)
4. Open the user -> **Security credentials** tab -> **Create access key** -> choose **Command Line Interface (CLI)** -> tick the confirmation -> Next -> Create.
5. Copy the **Access key ID** and **Secret access key** (or Download .csv). The secret is shown only once. **Never commit these to Git or share them.**

C3. Allow billing access for IAM users (needed for the Budget)

1. Sign in as **root** -> account name (top right) -> **Account**.
2. Scroll to **IAM user and role access to Billing information** -> **Edit** -> tick **Activate IAM Access** -> Update.

If you skip this, the stack may fail on `MonthlyBudget` with AccessDenied (see Part I).

C4. Install the AWS CLI v2

1. Download and run: https://awscli.amazonaws.com/AWSCLIV2.msi (Next, Next, Install).
2. **Close and reopen PowerShell** (required so the new PATH is loaded), go back to the project folder.
3. Check:

```
aws --version
```

Expected: `aws-cli/2.x.x Python/3.x Windows/...`

C5. Configure credentials

```
aws configure
```

Answer the four prompts:

```
AWS Access Key ID [None]: <paste access key id>
AWS Secret Access Key [None]: <paste secret access key>
Default region name [None]: ap-south-1
Default output format [None]: json
```

`ap-south-1` is Mumbai (closest to India). You may use another region, but then pass `-Region <region>` to every script below.

Verify:

```
aws sts get-caller-identity
```

Expected: JSON with your `Account` number and `"Arn": "arn:aws:iam::<account>:user/intern-deployer"`.

Part C problems

Problem	Fix
`aws is not recognized`	Reopen PowerShell after installing. Still failing: sign out/in to Windows, or check `C:\Program Files\Amazon\AWSCLIV2\` exists
`InvalidClientTokenId` / `SignatureDoesNotMatch`	Key copied wrongly (extra space). Run `aws configure` again. Also check the Windows clock is correct (Settings -> Time -> Sync now)
`Unable to locate credentials`	`aws configure` was not completed

Part D - Put the code on GitHub

The EC2 server downloads your code from GitHub and builds the Docker image itself, so the repository must be **public**.

D1. Create the repository

1. Sign in at https://github.com -> **+** (top right) -> **New repository**.
2. Name: `arrowstack-cloud-internship`. Visibility: **Public**. Do **not** add a README, .gitignore or license. -> **Create repository**.
3. Copy the HTTPS URL, e.g. `https://github.com/<your-username>/arrowstack-cloud-internship.git`.

D2. Push the whole internship folder

Push the parent folder `E:\ArrowStack Internship` (so the repo contains `project1-cloud-business-api/` - the deployment expects that path; Task 2 will be added next to it later).

```
cd "E:\ArrowStack Internship"
git init
git config user.name "Your Name"
git config user.email "you@example.com"
git add .
git status
```

Check the `git status` list: you should see `project1-cloud-business-api/...` files, and **no** `.venv`, `local.db` or `__pycache__` (they are ignored by `.gitignore`).

```
git commit -m "Project 1: BrightMart cloud-hosted business API"
git branch -M main
git remote add origin https://github.com/<your-username>/arrowstack-cloud-internship.git
git push -u origin main
```

A browser window may open asking you to sign in to GitHub - approve it. Refresh the GitHub page: you should see the folder `project1-cloud-business-api` containing `Dockerfile`.

Safety check: the repo contains **no secrets** - the API key and database password are generated inside AWS and never stored in code.

```
cd "E:\ArrowStack Internship\project1-cloud-business-api"
```

Part D problems

Problem	Fix
`remote origin already exists`	`git remote set-url origin <url>`
`Authentication failed` / 403	Sign in via the browser popup; or create a Personal Access Token (GitHub -> Settings -> Developer settings -> Tokens) and use it as the password
`src refspec main does not match any`	You have not committed yet - run the `git commit` line
`rejected ... fetch first`	The GitHub repo is not empty (you added a README). Run `git pull origin main --allow-unrelated-histories`, then push again
You pushed only the project1 folder as the repo root	That works too - add `-AppSubdir .` to the deploy command in E1

Part E - Deploy to AWS

E1. Run the deploy script

From `E:\ArrowStack Internship\project1-cloud-business-api`:

```
powershell -ExecutionPolicy Bypass -File .\scripts\deploy.ps1 `
  -GitRepoUrl https://github.com/<your-username>/arrowstack-cloud-internship.git `
  -AlertEmail <your-email>
```

(The backtick character at the end of a line means "the command continues on the next line" in PowerShell - paste all three lines together. `-ExecutionPolicy Bypass` lets this one script run without changing any Windows setting.)

You will see `Waiting for changeset to be created..` then `Waiting for stack create/update to complete`. **This takes 12-20 minutes** - the database alone takes about 10. Do not close the window.

E2. Watch progress (optional but recommended)

AWS Console -> make sure the region (top right) is **Asia Pacific (Mumbai)** -> search **CloudFormation** -> stack `brightmart-api` -> **Events** tab. Resources turn `CREATE_COMPLETE` one by one. The last ones are `AppInstance` (waits until the API answers `/health/ready`) and the alarms.

E3. Read the outputs

When finished, the script prints a table:

Output	Example	Meaning
ApiUrl	`http://13.200.x.x`	Your API
ApiDocsUrl	`http://13.200.x.x/docs`	Swagger UI
ApiKeySecretArn	`arn:aws:secretsmanager:...`	Where the API key lives
DashboardUrl	`https://ap-south-1.console.aws.amazon.com/cloudwatch/...`	Monitoring dashboard
InstanceId, DatabaseEndpoint, LogGroup		Used in later steps

Show them again any time:

```
aws cloudformation describe-stacks --stack-name brightmart-api --query "Stacks[0].Outputs[].[OutputKey, OutputValue]" --output table
```

Screenshot this table = evidence C1.

E4. Confirm the alert email

Check your inbox (and **spam**) for *"AWS Notification - Subscription Confirmation"* -> click **Confirm subscription**. Without this, alarm emails are not delivered.

Part F - Verify in the cloud and collect evidence

Create an evidence screenshot folder: `mkdir evidence\screenshots` (from the project folder).

F1. Get the API key

```
$key = aws secretsmanager get-secret-value --secret-id brightmart-api/api-key --query SecretString --
output text
$api = aws cloudformation describe-stacks --stack-name brightmart-api --query "Stacks[0].Outputs[
?OutputKey=='ApiUrl'].OutputValue" --output text
$api
```

(`$key` and `$api` live only in this PowerShell window. If you open a new window, run these two lines again.)

F2. Cloud smoke test (evidence C2)

```
.venv\Scripts\python scripts\smoke_test.py --base-url $api --api-key $key
```

Expected: `13/13 checks passed`. Screenshot it; the JSON report is saved in `evidence\`.

F3. Swagger in the cloud (evidence C3)

Open the ApiDocsUrl. **Type `http://` explicitly** - if the browser switches to `https://` the page will not load (there is no TLS; see limitations). Repeat the B4 walkthrough using the real key (`$key` - print it with `$key` if you need to paste it). Screenshot.

F4. Dashboard and logs (evidence C4, C5)

1. Open the DashboardUrl -> you should see requests, latency and CPU from your smoke test (metrics can take 3-5 minutes to appear). Screenshot.
2. Console -> **CloudWatch** -> **Logs Insights** -> select log group `/brightmart-api/app` -> paste:

```
fields @timestamp, method, path, status, duration_ms, request_id
| filter msg = "request"
| sort @timestamp desc
| limit 20
```

   -> **Run query**. Screenshot.

Live log tail from PowerShell: `aws logs tail /brightmart-api/app --follow` (Ctrl + C to stop).

F5. Alarm test (evidence C6)

```
aws cloudwatch set-alarm-state --alarm-name brightmart-api-5xx-errors --state-value ALARM --state-
reason "Manual alarm test"
```

Within ~1 minute you receive an email "ALARM: brightmart-api-5xx-errors". Screenshot it. The alarm returns to OK by itself.

F6. Security evidence (C7, C8, C9)

- **C7 IAM**: Console -> IAM -> Roles -> search `brightmart-api-InstanceRole` -> Permissions -> expand `app-runtime` -> screenshot (only 2 secrets + 1 log group).
- **C8 RDS**: Console -> RDS -> Databases -> `brightmart-api-db` -> **Connectivity & security** shows *Publicly accessible: No*; **Configuration** shows encryption enabled and the engine version; **Maintenance & backups** shows 7-day retention. Screenshots. Write the engine version into `docs/test-evidence.md`.
- **C9 DB not reachable from the internet**:

```
$db = aws cloudformation describe-stacks --stack-name brightmart-api --query "Stacks[0].Outputs[
?OutputKey=='DatabaseEndpoint'].OutputValue" --output text
Test-NetConnection $db -Port 5432
```

Expected: `TcpTestSucceeded : False` (good - the database is private). Screenshot.

F7. Readiness when the DB is down (evidence C11, optional, ~15 min)

```
aws rds stop-db-instance --db-instance-identifier brightmart-api-db
# wait ~5 minutes, then:
curl.exe -i "$api/health/ready"
```

Expected: `HTTP/1.1 503` and `"database":"unreachable"` while `/health` still returns 200. Then start it again and wait until it says `available`:

```
aws rds start-db-instance --db-instance-identifier brightmart-api-db
aws rds wait db-instance-available --db-instance-identifier brightmart-api-db
curl.exe -i "$api/health/ready"
```

Expected: 200 again - the app reconnects by itself.

F8. Restore drill - measure your RTO (evidence C10, optional, ~30 min, small cost)

Follow `docs/operations-runbook.md` -> "Restore drill". Short version:

```
Get-Date   # T0
aws rds restore-db-instance-to-point-in-time `
  --source-db-instance-identifier brightmart-api-db `
  --target-db-instance-identifier brightmart-api-db-restored `
  --use-latest-restorable-time `
  --db-instance-class db.t4g.micro `
  --no-publicly-accessible `
  --db-subnet-group-name <DbSubnetGroup id> `
  --vpc-security-group-ids <DbSecurityGroup id>
aws rds wait db-instance-available --db-instance-identifier brightmart-api-db-restored
Get-Date   # T1  -> RTO = T1 - T0
aws rds delete-db-instance --db-instance-identifier brightmart-api-db-restored --skip-final-snapshot
```

The two IDs come from CloudFormation -> stack -> **Resources** tab (`DbSubnetGroup` and `DbSecurityGroup` rows, "Physical ID" column). Note: two DB instances running at once uses free-tier hours twice as fast - delete the restored one immediately.

F9. Budget (evidence C12)

Console -> **Billing and Cost Management** -> **Budgets** -> `brightmart-api-monthly` visible. Screenshot.

F10. Record results

Fill in the table in `docs/test-evidence.md` section 4 and add a dated line to `docs/worklog.md`. Commit and push:

```
git add .
git commit -m "Add cloud validation evidence"
git push
```

Part G - Ship a code change (optional)

1. Edit code, run the tests (B2).
2. `git add .` -> `git commit -m "..."` -> `git push`.
3. Redeploy only the app (no infrastructure change, ~1-2 minutes):

```
powershell -ExecutionPolicy Bypass -File .\scripts\redeploy_app.ps1
```

Expected: JSON with `"Status": "Success"` and `{"status":"ok","database":"reachable"}` in the output.

Shell on the server (no SSH needed): `aws ssm start-session --target <InstanceId>` (requires the Session Manager plugin: https://s3.amazonaws.com/session-manager-downloads/plugin/latest/windows/SessionManagerPluginSetup.exe). Inside: `sudo docker ps`, `sudo docker logs api --tail 50`. Type `exit` to leave.

Part H - Tear down (do this when your demo/review is finished)

```
powershell -ExecutionPolicy Bypass -File .\scripts\teardown.ps1
```

Type `brightmart-api` to confirm. It takes ~10-15 minutes (RDS takes a final snapshot first). Then:

1. The script lists remaining snapshots. Delete them when you no longer need the data:

   `aws rds delete-db-snapshot --db-snapshot-identifier <id>`

2. The API key secret is kept in a recovery window. To delete it immediately:

   `aws secretsmanager delete-secret --secret-id brightmart-api/api-key --force-delete-without-recovery`

3. Billing check after 24 hours: Console -> **Billing** -> **Bills** -> current month should be ~$0.
4. After the internship: delete the IAM user `intern-deployer` (or at least its access key).

The Budget is deleted with the stack.

Part I - Troubleshooting

I1. How to find the real error when a deploy fails

1. CloudFormation -> stack -> **Events** tab -> sort by time -> find the **first** row with status `CREATE_FAILED` (ignore later "Resource creation cancelled" rows). Its **Status reason** is the real error.
2. If the stack ends in `ROLLBACK_COMPLETE`, it cannot be updated - delete it first, then deploy again:

```
aws cloudformation delete-stack --stack-name brightmart-api
aws cloudformation wait stack-delete-complete --stack-name brightmart-api
```

I2. Deploy errors

Error (Status reason)	Cause	Fix
`AppInstance: Failed to receive 1 resource signal(s) within the specified duration`	The server could not download/build/start the app	See I3
`Requires capabilities : [CAPABILITY_IAM]`	Ran `aws cloudformation deploy` by hand	Use `deploy.ps1` (it adds the flag)
`MonthlyBudget: ... AccessDenied`	IAM billing access not activated	Do step C3, delete the stack (I1), deploy again
`... is not authorized to perform: iam:CreateRole`	IAM user lacks permissions	Attach AdministratorAccess to `intern-deployer` (C2)
`DBInstance brightmart-api-db already exists`	Old stack/DB with the same name	Delete the old DB/stack, or deploy with `-StackName brightmart-api2` (and use that name in all commands)
`brightmart-api/api-key already exists` / `scheduled for deletion`	Secret from a previous attempt	`aws secretsmanager delete-secret --secret-id brightmart-api/api-key --force-delete-without-recovery`, then deploy again
`The requested configuration is currently not supported` / `InsufficientInstanceCapacity` / instance type not free-tier eligible	t3.micro unavailable in that AZ/region/plan	Add `-InstanceType t2.micro` (or use another region with `-Region`)
`DB instance class ... not supported` / free plan restriction on RDS	Plan or region limitation	Edit `infra/template.yaml` parameter `DBInstanceClass` to `db.t3.micro` and redeploy
`Parameter AlertEmail failed to satisfy constraint`	Typo in email	Correct the `-AlertEmail` value
`The maximum number of addresses has been reached`	5 Elastic IPs already used in region	Release unused EIPs (EC2 -> Elastic IPs)
`No changes to deploy`	Nothing changed	Not an error
`running scripts is disabled on this system`	PowerShell policy	Use `powershell -ExecutionPolicy Bypass -File ...` as shown
Script stops with `Waiter StackCreateComplete failed`	Generic: the stack failed	Go to I1

I3. The app did not start on the server (resource signal failure)

Most common causes, in order:

1. **Repository is private** or URL is wrong -> make it public / fix the URL (open it in a browser in a private window to test).
2. **Branch name** is `master`, not `main` -> add `-GitBranch master`.
3. **Folder layout**: the repo root must contain `project1-cloud-business-api/Dockerfile`. If the Dockerfile is at the repo root, add `-AppSubdir .`.

4. Docker Hub download limit (`toomanyrequests`) -> wait 30 minutes and deploy again.

To read the server's boot log, deploy with rollback disabled so the instance is kept:

```
aws cloudformation delete-stack --stack-name brightmart-api
aws cloudformation wait stack-delete-complete --stack-name brightmart-api
powershell -ExecutionPolicy Bypass -File .\scripts\deploy.ps1 -GitRepoUrl <url> -AlertEmail <email> -
NoRollback
```

When it fails: EC2 console -> select the instance `brightmart-api-app` -> **Actions -> Monitor and troubleshoot -> Get system log**, scroll to the end; or connect with Session Manager (Part G) and run `sudo tail -100 /var/log/cloud-init-output.log`. Fix the cause, then delete the stack (I1) and deploy again normally.

I4. Problems after a successful deploy

Symptom	Cause	Fix
Browser: "This site can't be reached" / timeout	Browser forced `https://`, or your network blocks port 80, or `-AllowedHttpCidr` excludes your IP	Type `http://<ip>/docs` explicitly; try mobile hotspot; redeploy with `-AllowedHttpCidr 0.0.0.0/0`
`/health` 200 but `/health/ready` 503	DB stopped/starting, or credentials problem	RDS console: status must be *Available*. Then check logs: `aws logs tail /brightmart-api/app --since 30m`
401 on POST in the cloud	Using `local-dev-key`	Use the real key from F1 (`$key`)
Smoke test `$api` is empty	Variables lost (new window)	Re-run the two lines in F1
No alarm email	Subscription not confirmed / in spam	E4; or SNS console -> Subscriptions -> Request confirmation
Dashboard empty	Metrics delay	Wait 5 minutes after generating traffic; set the time range to 1h
`aws ssm start-session`: SessionManagerPlugin is not found	Plugin not installed	Install the plugin (link in Part G), reopen PowerShell
`redeploy_app.ps1` status `Failed`	Build or readiness failed	Read the `Errors` text in its output; test locally first (B2-B5)

I5. Teardown problems

Symptom	Fix
`DELETE_FAILED`	Run `teardown.ps1` again; if a resource is still stuck, delete it manually in its console, then retry
Still being charged	Billing -> Bills -> look per service; check RDS snapshots, Elastic IPs and the region selector (resources in another region)

I6. Getting help

When something fails, send me:

1. The exact command you ran.
2. The full error text (copy-paste, not a photo if possible).
3. For deploy failures: the **first** `CREATE_FAILED` row from CloudFormation -> Events.

Final submission checklist

- [ ] `pytest` 35 passed, coverage report in `evidence/pytest-report.txt`
- [ ] Local smoke test 13/13 (`evidence/smoke-*.json`)
- [ ] Cloud smoke test 13/13 + screenshots C1-C12 in `evidence/screenshots/`
- [ ] `docs/test-evidence.md` section 4 filled in, `docs/worklog.md` updated
- [ ] GitHub repo public and up to date, README renders correctly
- [ ] Presentation prepared from `docs/presentation-outline.md`
- [ ] Stack torn down after the review, billing checked